
CSE 151B/251B Milestone Report

github.com/beepbeepimajee/151B_SP26_Competition

Rain van Eetveldt

University of California, San Diego
dvaneetv@ucsd.edu

Abstract

This project fine-tunes Qwen3-4B-Thinking-2507 with QLoRA SFT for a Kaggle math reasoning competition with mixed multiple-choice and free-form questions. The current best Kaggle public leaderboard score is 0.614, achieved with the base model + N=8 self-consistency on a v1-baseline prompt. A subsequent supervised fine-tuning attempt on 16k OpenR1 chain-of-thought traces produced a model that trains cleanly (eval loss 0.4035, monotonically decreasing) but exhibits repetition collapse in long generation at default sampling — a failure invisible to training metrics. A repetition_penalty adjustment recovers most of the model’s capability and is the next experimental step.

1 Introduction

Problem Definition. The task is mathematical reasoning on mixed-format problems: 1126 questions split between multiple-choice (with up to 9 options) and free-form numerical/symbolic answers. The model receives a problem statement and must produce a final answer inside `\boxed{}`. Grading is exact-match on the final answer; reasoning quality is not directly scored.

Problem Significance. Competition mathematics is a benchmark for compositional reasoning, multi-step inference, and symbolic manipulation. Strong performance on these tasks has applications in tutoring systems, automated theorem proving, and scientific computing. Compact reasoning models (4B parameters) that approach larger-model performance also matter for deployment economics.

Technical Challenge. Three challenges: (1) Data — most public math datasets contain answer keys, not chain-of-thought reasoning, so SFT data must be carefully sourced. (2) Compute — fine-tuning a 4B model on long sequences (max 11k tokens) on a single 24GB GPU requires careful memory management (gradient checkpointing, activation memory tradeoffs). (3) Evaluation — local accuracy on held-out slices is a noisy proxy for Kaggle score; format mismatches between local judger and Kaggle grader can produce dramatic local-vs-leaderboard gaps.

Contributions.

- Established a strong baseline (Kaggle 0.614) using base Qwen3-Thinking-2507 with N=8 self-consistency, no fine-tuning.
- Identified and documented a category-error failure mode: NuminaMath solutions are answer keys, not reasoning chains, and SFT on this data produced a 50pp local accuracy regression invisible to eval-loss curves.
- Sourced 16k verified-correct chain-of-thought traces from OpenR1-Math-220k with multi-stage filtering (correctness verification, single-answer enforcement, length filtering, parseable boxed answer requirement).

- Diagnosed an SFT-induced repetition collapse pathology in long generation, fixable via `repetition_penalty=1.1` sampling adjustment.
- Documented three independent SFT failure modes invisible to training metrics, contributing to a methodological insight: eval-loss on training-distribution holdout does not validate output quality on the deployment task.

2 Methods

Problem Setting. The dataset contains 1126 questions in `public.jsonl` (with gold answers) and 943 questions in `private.jsonl` (Kaggle test set, gold withheld). Each question has fields `question` (problem text), `options` (list, present for MCQ), and `answer` (string, present in public). The model takes a system prompt + user prompt as input and produces a sequence of tokens; the final answer is extracted from the last `\boxed{...}` expression. Training optimizes the standard cross-entropy loss on assistant tokens only, with the system and user portions masked.

Idea Summary. Two complementary approaches: (1) inference-time methods that improve answer quality without modifying the model — self-consistency with majority voting [3] (N=8 samples per question) reduces single-sample variance; (2) supervised fine-tuning on chain-of-thought reasoning traces to teach the model the reasoning style of strong math models (DeepSeek-R1 outputs in OpenR1-Math-220k). The fine-tuning approach risks degrading the base model’s general reasoning capability if training data is mis-specified or if SFT induces output pathologies; mitigations include systematic data inspection, eval-loss monitoring, and inference-time validation on held-out task slices.

3 Experiments

3.1 Baselines

- **Base Qwen3-Thinking-2507 single-sample:** Run 08 on full 943 private set with v1-baseline prompt and the model’s vendor sampling defaults (temperature=0.6, top_p=0.95, top_k=20). Kaggle public LB 0.586.
- **Base Qwen3-Thinking-2507 + SC-8:** Run 09 with N=8 self-consistency majority vote. Kaggle public LB 0.614.
- **v3-perslot prompt variant:** Run 10, same model with a per-slot multi-answer prompt (`\boxed{a}` `\boxed{b}` for paired answers). Kaggle public LB 0.424 (-16.2pp regression vs Run 08), establishing that comma-joined single-box format is required by the Kaggle grader.

Table 1: Kaggle public leaderboard scores across all four submissions to date. Run 09 (base Qwen3-Thinking-2507 with N=8 self-consistency) is the best result. The Run 09 local-accuracy column shows a parser anomaly specific to the SC majority-vote output format; Kaggle’s grader scored the same outputs at 0.614, isolating the discrepancy to the local scorer rather than the model. All four submissions used the full 943-question private set.

#	Date	Run / method	Local acc.	Kaggle LB
1	2026-05-04	Run 08-v2 — base, single sample, v1-baseline	N/A	0.586
2	2026-05-05	Run 09 — base + N=8 self-consistency, v1-baseline	33.2%*	0.614
3	2026-05-04 / 05	Run 10 — base, v3-perslot multi-answer prompt	N/A	0.424
4	2026-05-05	Experiment A — Run 08 outputs reformatted per-slot	N/A	0.420

*Local-judger anomaly: SC majority-vote MCQ output is parsed as 0/300 correct by the local scorer, but Kaggle’s grader scored the same outputs at 0.614. Investigation pending.

3.2 Evaluation

Three evaluation tiers: (1) Kaggle public leaderboard score — primary metric, but limited to 3 submissions per day. (2) Local accuracy on `fixed_50_v1`, a 50-question stratified slice (17 MCQ +

33 free-form) of the public set, computed via a local judger that approximates the Kaggle grader’s exact-match logic. (3) During-training eval/assistant_only_loss on the last 8 rows of the training file, computed every 250 steps. The 50-question slice has an estimated noise floor of ± 5 questions, giving a +4-question promotion threshold (39/50) for clean wins over baselines.

3.3 Implementation Details

Training and inference run on a single RTX 4090 (24GB VRAM) on RunPod, AMD EPYC 7B13 CPU. Software: PyTorch 2.6.0+cu124, Unsloth 2026.5.2 for QLoRA fine-tuning, vLLM 0.8.5 for inference. Two virtual environments are maintained — one with transformers 5.5.0 for training, one with transformers 4.51.3 for vLLM inference.

Fine-tuning hyperparameters: Qwen3-4B-Thinking-2507 [2] loaded in 4-bit [1] via Unsloth, LoRA rank 32, alpha 32, target modules all linear layers (q/k/v/o + gate/up/down). Effective batch size 8 (per-device 1 + gradient accumulation 8). Max sequence length 11000 (covering p95=10450 of training data). Learning rate 2e-4, cosine schedule with 5% warmup. AdamW 8-bit optimizer. Gradient checkpointing enabled (Unsloth variant) to fit activation memory at long sequences. 1500 training steps on 16k OpenR1 examples, completing in 10h 33m wall-clock at 25.6 sec/step.

Inference hyperparameters: vLLM BF16, max_new_tokens=16384, max_model_len=24576, sampling per Qwen3-Thinking-2507 vendor recommendations. Single-sample inference on fixed_50_v1 runs in ~ 21 minutes; SC-8 inference on the full private set runs ~ 28 hours.

3.4 Results

Result 1 — Self-consistency lifts Kaggle score by 2.8pp over single-sample base model. Run 08 (base model, single sample) scored 0.586 on Kaggle public LB; Run 09 (base model, SC-8) scored 0.614, a +2.8pp lift. SC infrastructure is the strongest result to date. The full submission record is shown in Table 1.

Result 2 — Format errors dominate model errors. Run 10’s per-slot multi-answer format produced a 16.2pp Kaggle regression (0.424 vs Run 08’s 0.586) despite no model or reasoning change. Experiment A confirmed this by reformatting Run 08’s responses to per-slot and re-submitting — score dropped to 0.420, isolating the format effect at -16.6pp.

Result 3 — Eval loss does not predict output quality. A v2 SFT run on NuminaMath (8k examples, eval loss 0.5131 monotonically decreasing) produced a model with 20% local accuracy versus the base model’s 70% on the same slice. Inspection of the training data revealed that NuminaMath solutions are answer keys (1200 char averaged paraphrased problem restatements), not chain-of-thought reasoning. The model fit cleanly to the wrong-category data with no signal from training metrics.

Result 4 — SFT can induce sampling-recoverable generation pathologies. A v2 OpenR1 SFT run (16k examples, final eval/assistant_only_loss = 0.4035 with monotonic descent throughout training, Figure 1) at inference time exhibited repetition collapse: 30 of 50 outputs failed to produce a parseable answer because generation degenerated into token loops (e.g., repeated system tokens, meaning meaning meaning...) before reaching `\boxed{}`. Figure 2 shows the tail of one such collapsed output. A targeted re-run with repetition_penalty=1.1 (other parameters unchanged) recovered 2 of 3 sampled failures with 86–90% reduction in generation length and clean parseable output, indicating the pathology is sampling-related rather than knowledge degradation.

4 Discussion

Achievements. A best Kaggle public LB score of 0.614 using base model + SC-8. A reproducible end-to-end pipeline for QLoRA fine-tuning Qwen3-4B-Thinking-2507 on a single 24GB GPU, including data prep, training, adapter merging, and inference. Documentation of three SFT failure modes invisible to training metrics, contributing methodological insight relevant beyond this project.

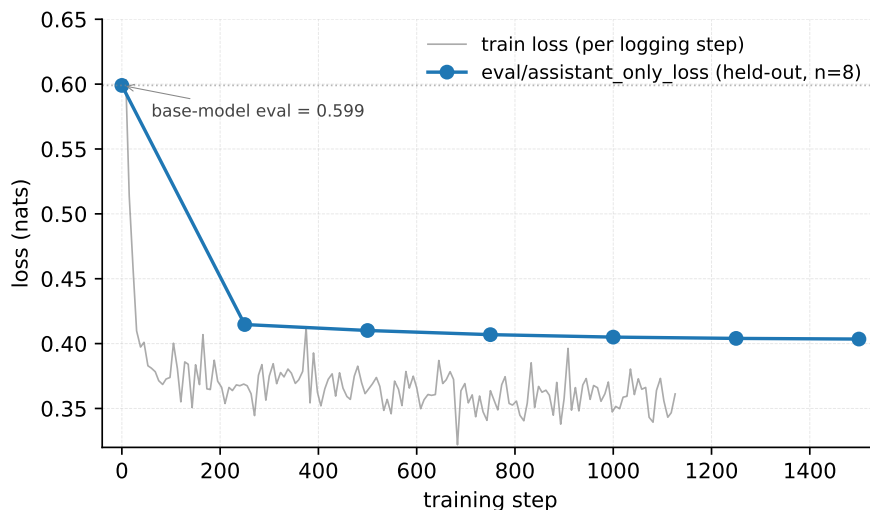


Figure 1: v2 OpenR1 training curves over 1,500 steps (16k examples, single epoch). Train loss (gray, every 10 steps) and held-out eval/assistant_only_loss (blue, every 250 steps) both descend monotonically; the eval loss falls from a base-model anchor of 0.599 nats to 0.4035 at step 1500 with no sign of overfitting. By every standard training-time health signal, this run is clean. The inference-time collapse pathology described in Result 4 (Figure 2) was not visible in any of these curves.

```
as and,systemssystemsystem,systemsystem, \.systemtextssystemssystemsystem
msystemssystemsystem.system..systemssystemssystemssystemssystemsystem
systemssystemssystemssystemssystemssystemssystemssystemssystemsystem
```

Figure 2: Final 200 characters of the v2 OpenR1 model’s generation on fixed_50_v1 item id=53 (an MCQ integral, run11_v2openr1_50_tok16384.jsonl). Generation reached the 16,384-token cap without producing a parseable `\boxed{}`; the tail shows the token-repetition mode that 30 of 50 Run 11 outputs degenerated into. (Line breaks above are inserted for display; the original output is one continuous string with no whitespace in the repeating section.) The collapse begins after several thousand tokens of nominally coherent reasoning and is invisible in the training curves of Figure 1.

Bottlenecks and Mitigation. The critical bottleneck has been data quality and inference reliability rather than model capacity. NuminaMath’s data category mismatch and OpenR1’s generation collapse both passed all training-time health checks, surfacing only at inference. Mitigations: systematic inspection of training data samples (catching NuminaMath’s answer-key format), and inference validation on held-out task slices (catching OpenR1’s collapse). Going forward, mid-training inference passes on a fixed task slice will replace eval-loss as the primary health signal.

Plans Forward. Two near-term experiments: (1) re-run v2 OpenR1 inference with `repetition_penalty=1.1` to measure the model’s actual capability after the sampling fix; if competitive with the base model + SC-8 baseline, pursue v2 OpenR1 + SC-8 as the next submission candidate. (2) Inference-time improvements independent of model choice: cutoff-triggered retry with larger token budget to recover the 5–12% of items that hit the `max_new_tokens` cap. Longer-term, if v2 OpenR1 underperforms after the sampling fix, the data prep pipeline will be redesigned for v3 with stricter generation-quality validation built into mid-training checkpoints.

Team Responsibilities. Solo project. Rain van Eetveldt — undergraduate computer science student at UC San Diego — is responsible for all aspects: literature survey, data prep, training infrastructure, model fine-tuning, inference pipeline, evaluation, and writeup.

Timelines.

- Weeks 2–4: Project framing, infrastructure setup, baseline runs (Runs 01–06). Established Kaggle pipeline and v1-baseline prompt.
- Weeks 5–6: Self-consistency exploration (Runs 07, 09). Established Kaggle 0.614 baseline.
- Week 7: Format diagnostic (Run 10, Experiment A). Established that Kaggle grader requires comma-joined single-box format.
- Week 8 (current): SFT experiments. v2 NuminaMath (failed, post-mortem complete). v2 OpenR1 (collapse diagnosed, sampling fix identified).
- Week 9: v2 OpenR1 + repetition_penalty rerun. If successful, SC-8 submission candidate. Cutoff-retry inference improvements.
- Week 10: Final report writeup. Decision between v2 OpenR1+SC-8 submission and v3 retrain depending on Week 9 results.

LLM Usage

LLMs were used extensively as a general-purpose research assistant throughout this project: for ideation, code review, debugging assistance, technical research synthesis (e.g., a multi-agent prompt engineering literature dive), and writing assistance for this report. The author takes full responsibility for all content and claims. No LLM is listed as a contributor.

References

- [1] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [2] Qwen Team. Qwen3 technical report. <https://qwenlm.github.io/blog/qwen3/>, 2025.
- [3] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.